

A **handler function** in Go is simply a function that **receives an HTTP request and writes an HTTP response**.

When a client (browser, Postman, mobile app, etc.) sends a request to your server, Go calls the appropriate handler function.

Basic Syntax

```
func handler(w http.ResponseWriter, r *http.Request) {  
    fmt.Fprintln(w, "Hello, World!")  
}
```

The parameters are:

- **w http.ResponseWriter** → Used to send data back to the client.
 - **r *http.Request** → Contains everything about the incoming request (URL, method, headers, body, etc.).
-

Example

```
package main  
  
import (  
    "fmt"  
    "net/http"  
)  
  
func helloHandler(w http.ResponseWriter, r *http.Request) {  
    fmt.Fprintln(w, "Hello from Go!")  
}  
  
func main() {  
    http.HandleFunc("/hello", helloHandler)  
  
    fmt.Println("Server running on :8080")  
    http.ListenAndServe(":8080", nil)  
}
```

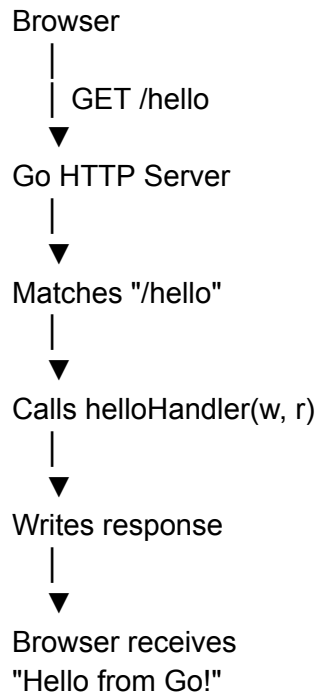
When someone visits:

`http://localhost:8080/hello`

Go executes

helloHandler(w, r)

Flow



Why is it called a "handler"?

Because it **handles** an incoming HTTP request.

For example,

GET /users

might be handled by

```
func getUsers(w http.ResponseWriter, r *http.Request) {  
  
}
```

and

POST /users

might be handled by

```
func createUser(w http.ResponseWriter, r *http.Request) {  
  
}
```

Each handler is responsible for one endpoint (or one group of requests).

What can a handler do?

A handler can:

Read the request:

```
name := r.URL.Query().Get("name")
```

Read JSON:

```
var user User  
json.NewDecoder(r.Body).Decode(&user)
```

Check HTTP method:

```
if r.Method != http.MethodPost {  
    http.Error(w, "Method Not Allowed", http.StatusMethodNotAllowed)  
    return  
}
```

Return JSON:

```
w.Header().Set("Content-Type", "application/json")  
json.NewEncoder(w).Encode(user)
```

Set status code:

```
w.WriteHeader(http.StatusCreated)
```

Why **HandleFunc**?

Go provides:

```
http.HandleFunc("/hello", helloHandler)
```

This registers the function for the **/hello** route.

Internally it means:

If request URL == **/hello**

↓

Call helloHandler()

A real backend example

```
func createStudent(w http.ResponseWriter, r *http.Request) {  
  
    if r.Method != http.MethodPost {  
        http.Error(w, "Only POST allowed", http.StatusMethodNotAllowed)  
        return  
    }  
  
    var student Student  
  
    err := json.NewDecoder(r.Body).Decode(&student)  
    if err != nil {  
        http.Error(w, "Invalid JSON", http.StatusBadRequest)  
        return  
    }  
  
    // Business logic  
    student.ID = 1  
  
    w.Header().Set("Content-Type", "application/json")  
}
```

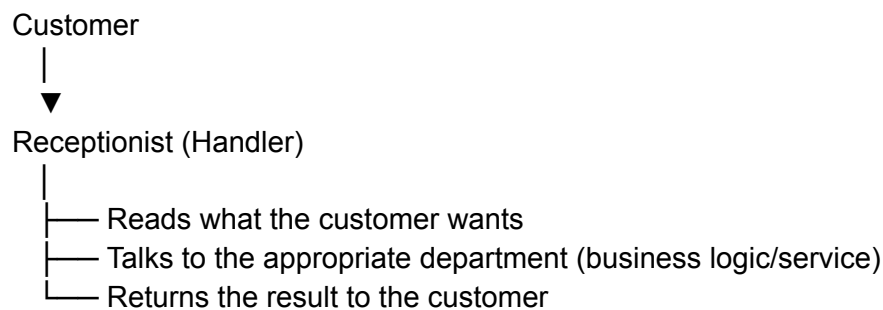
```
    json.NewEncoder(w).Encode(student)
}
```

Here the handler performs several responsibilities:

1. Receives the request.
 2. Validates the HTTP method.
 3. Reads the JSON body.
 4. Executes business logic (or calls a service that contains it).
 5. Sends the response.
-

Think of it like a receptionist

Imagine a company office:



In a well-structured Go application, the handler should usually stay **thin**. It handles HTTP-specific work (reading the request and writing the response) and delegates the actual business rules to a service layer. This separation makes the code easier to test, maintain, and reuse.

This is actually one of the most important lines you'll write in Go web development. Let's break it down **step by step**, including what happens inside the Go standard library.

```
err := json.NewDecoder(r.Body).Decode(&student)
```

Think of it as three separate operations:

```
r.Body
|
```

▼
json.NewDecoder(r.Body)
|
▼
Decoder.Decode(&student)

Let's understand each one.

Step 1: **r.Body**

```
func(w http.ResponseWriter, r *http.Request)
```

The **r** is the incoming HTTP request.

Suppose a client sends this request:

```
POST /students
Content-Type: application/json
```

```
{
  "name": "Suyash",
  "age": 23
}
```

Go stores that JSON inside the request body.

Request

- Method = POST
- URL = /students
- Headers
- Body

▼

```
{
  "name": "Suyash",
  "age": 23
}
```

So,

`r.Body`

is a stream from which Go can read the request body.

Its type is

`io.ReadCloser`

That means it behaves like a file—you read it sequentially.

Step 2: `json.NewDecoder(r.Body)`

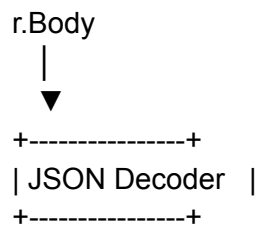
Now Go says,

"I know this body is just bytes. I need something that understands JSON."

So you create a decoder.

```
decoder := json.NewDecoder(r.Body)
```

Internally, you can think of it like:



The decoder knows how to convert JSON into Go structs.

Notice:

```
NewDecoder(...)
```

does **not** read anything yet.

It only prepares the decoder.

Think of it like opening a book.

Book

|

Open Book

|

Still haven't read it.

Step 3: `.Decode(&student)`

Now the decoder actually starts reading.

```
decoder.Decode(&student)
```

It:

1. Reads bytes from `r.Body`
 2. Parses the JSON
 3. Finds matching struct fields
 4. Stores the values inside `student`
-

Suppose

```
type Student struct {  
    Name string `json:"name"`  
    Age  int   `json:"age"`  
}
```

Incoming JSON

```
{  
  "name": "Suyash",  
  "age": 23  
}
```


Initially

student

Name = ""

Age = 0

After decoding

student

Name = "Suyash"

Age = 23

That's why we pass

&student

and not

student

because the decoder needs to **modify** the original variable. Passing `&student` gives it the memory address where it can write the decoded values.

Why `&student`?

Imagine:

student

Address: 0x1234

Passing

student

gives the decoder only a copy.

Passing

&student

gives it:

0x1234

Now it can write directly into that memory location.

What does `NewDecoder()` return?

It returns a pointer to a JSON decoder.

```
decoder := json.NewDecoder(r.Body)
```

Type:

```
*json.Decoder
```

Think of it like an object with methods:

```
decoder.Decode(...)
```

```
decoder.Token()
```

```
decoder.More()
```

You're using its `Decode` method.

Why chain them together?

Instead of writing:

```
decoder := json.NewDecoder(r.Body)
```

```
err := decoder.Decode(&student)
```

Go programmers often write:

```
err := json.NewDecoder(r.Body).Decode(&student)
```

because it's shorter and the decoder is only needed once.

What is this checking?

```
if errors.Is(err, io.EOF) {  
}
```

`io.EOF` stands for **End Of File**.

In this context, it usually means the request body was **empty**.

For example:

POST /students

(no JSON body)

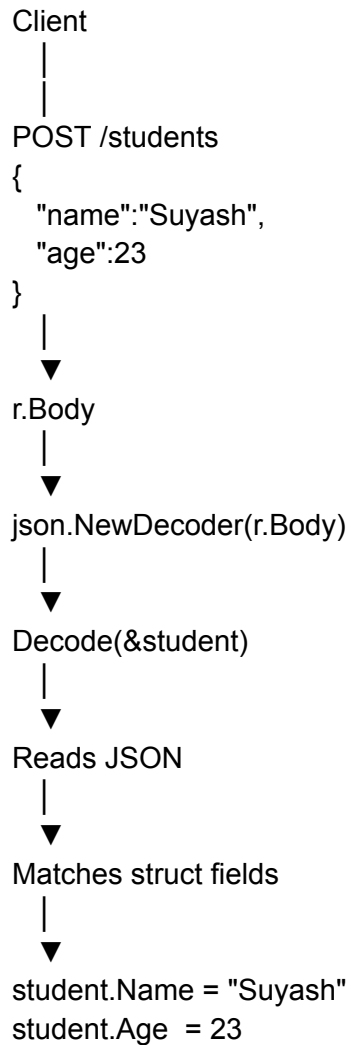
When the decoder tries to read, it immediately reaches the end of the stream, so it returns:

```
err == io.EOF
```

You typically handle it like this:

```
if errors.Is(err, io.EOF) {  
    http.Error(w, "Request body is empty", http.StatusBadRequest)  
    return  
}
```

Complete flow



This single line:

```
err := json.NewDecoder(r.Body).Decode(&student)
```

is effectively saying:

"Take the JSON sent by the client, read it from the request body, convert it into a Go **Student** struct, store the result in **student**, and tell me if anything went wrong."

This is a **great question**. Once you understand this, you'll understand how Go's HTTP server actually works.

Let's go one line at a time.

```
func WriteJSON(w http.ResponseWriter, status int, data interface{}) error {  
  
    w.Header().Set("Content-Type", "application/json")  
    w.WriteHeader(status)  
  
    return json.NewEncoder(w).Encode(data)  
}
```

First, what is this function?

It's just a **helper function**.

Instead of writing this in every handler:

```
w.Header().Set("Content-Type", "application/json")  
w.WriteHeader(http.StatusOK)
```

```
json.NewEncoder(w).Encode(student)
```

you simply write

```
WriteJSON(w, http.StatusOK, student)
```

Much cleaner.

Line 1

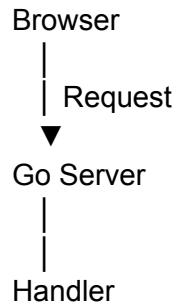
```
func WriteJSON(  
    w http.ResponseWriter,  
    status int,  
    data interface{},  
) error
```

It takes three inputs.

1. `w` `http.ResponseWriter`

This is **where you send the response back to the client**.

Imagine this:



The handler receives two things.

Request -----> `r`
Response (empty) -----> `w`

The browser is waiting for you to fill the response.

`w` is that response.

Initially it's empty.

HTTP Response

Headers:

Body:

Status:

Your job is to fill it.

Line 2

```
w.Header().Set("Content-Type", "application/json")
```

Every HTTP response has headers.

Example:

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 53

This line tells the browser

"I'm sending JSON."

Without this, the browser doesn't know what format the body is.

Now the response becomes

Status:

Headers:

Content-Type = application/json

Body:

Line 3

```
w.WriteHeader(status)
```

Now you're writing the HTTP status code.

Example

```
WriteJSON(w, 200, student)
```

becomes

HTTP/1.1 200 OK

If

WriteJSON(w, 404, ...)

then

HTTP/1.1 404 Not Found

Now response becomes

Status:

200

Headers:

Content-Type: application/json

Body:

Still empty body.

Last line

```
return json.NewEncoder(w).Encode(data)
```

This is the opposite of what you learned before.

Earlier you did

```
json.NewDecoder(r.Body).Decode(&student)
```

That means

JSON



Go Struct

Now we're doing


```
json.NewEncoder(w).Encode(student)
```

which means

Go Struct



JSON

Suppose

```
student := Student{
    Name: "Suyash",
    Age: 23,
}
```

The encoder converts it into

```
{
  "name": "Suyash",
  "age": 23
}
```

and writes it directly into `w`.

Why `NewEncoder(w)`?

Remember,

`w` is where responses are written.

The encoder needs somewhere to write JSON.

So

```
json.NewEncoder(w)
```

means

"Create something that can write JSON into the HTTP response."

Then

Encode(student)

actually writes it.

Complete flow

Student Struct

Name = "Suyash"

Age = 23



JSON Encoder



```
{  
  "name": "Suyash",  
  "age": 23  
}
```



HTTP ResponseWriter (w)



Browser

Now your second question

Why is it `w http.ResponseWriter` but `r *http.Request`?

This is one of the best questions a Go beginner can ask.

The reason is that they are used differently.

Request

`r *http.Request`

Notice the `*`.

That means

pointer to Request.

The request already exists.

The Go HTTP server creates it.

Browser



HTTP Request



Go Server

Go creates one big `Request` object.

Instead of copying the entire request, Go gives you a pointer.

Imagine the request contains

Headers

Cookies

Body

Method

URL

Context

Multipart files

That's a lot of data.

Copying it would be expensive.

So Go gives

Address
0x1234

instead.

Much faster.

ResponseWriter

Now look at

`http.ResponseWriter`

No `*`.

Why?

Because **ResponseWriter** is not a struct.

It is an **interface**.

Look at its definition (simplified):

```
type ResponseWriter interface {  
    Header() Header  
    Write([]byte) (int, error)  
    WriteHeader(statusCode int)  
}
```

Notice

interface

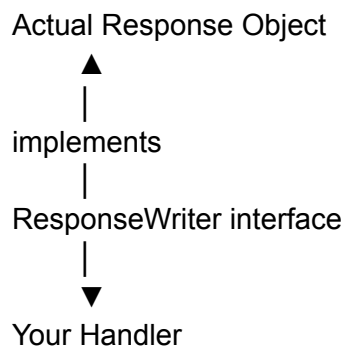
not

struct

Interfaces in Go are already small values that internally point to the actual implementation. Passing a pointer to an interface (`*http.ResponseWriter`) is almost never what you want.

Behind the scenes, Go creates a concrete object that implements this interface, and your handler receives the interface value.

Think of it like this:



You don't care what the concrete type is. You only care that it has the methods `Header`, `Write`, and `WriteHeader`.

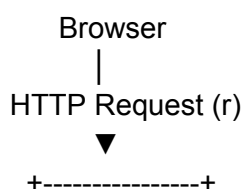
A simple rule to remember

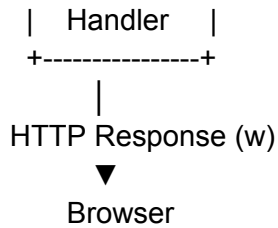
- `*http.Request` → A pointer to a large request struct created by the server. You read information from it (and sometimes modify parts like its context).
- `http.ResponseWriter` → An interface that lets you write the HTTP response. It already behaves like a reference to the underlying response object.

So when you see these two together:

```
func handler(w http.ResponseWriter, r *http.Request)
```

think:





- **r** is **what the client sent to you**.
- **w** is **what you'll send back to the client**.

Once this mental model clicks, almost every Go HTTP handler becomes much easier to understand.

Exactly! You're thinking in the right direction. Let me refine it a little because the terminology matters.

***http.Request**

This is a **pointer to a struct** (an object, if you're thinking in OOP terms).

Simplified:

```
type Request struct {  
    Method string  
    URL    *url.URL  
    Header Header  
    Body   io.ReadCloser  
    // many more fields...  
}
```

Your handler receives:

```
r *http.Request
```

So **r** points to an actual **Request** object that Go created.

You can access its fields:

```
r.Method  
r.URL.Path  
r.Header
```

r.Body

http.ResponseWriter

This is an **interface**, which is indeed an **abstraction**.

Simplified:

```
type ResponseWriter interface {
    Header() Header
    Write([]byte) (int, error)
    WriteHeader(statusCode int)
}
```

Notice there is **no implementation**, only a contract.

It says:

"Any type that has these three methods can be used as a ResponseWriter."

Who implements it?

The Go HTTP server has its own internal type (something like this):

```
type response struct {
    // internal fields
}

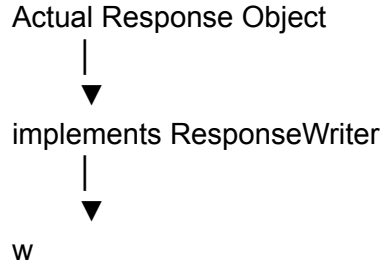
func (r *response) Header() Header { ... }
func (r *response) Write([]byte) (int, error) { ... }
func (r *response) WriteHeader(statusCode int) { ... }
```

Because it implements all the required methods, it satisfies the **ResponseWriter** interface.

So when your handler runs:

```
func handler(w http.ResponseWriter, r *http.Request)
```

what actually happens is roughly:



You don't know (or need to know) the concrete type. You just know it can write headers, status codes, and the response body.

Java analogy

Since you know Java, think of it like this:

```
interface Animal {  
    void speak();  
}  
  
class Dog implements Animal {  
    public void speak() {  
        System.out.println("Woof");  
    }  
}
```

Then:

```
Animal a = new Dog();
```

Here:

- **Animal** → Interface (abstraction)
- **Dog** → Concrete object

Similarly in Go:

```
var w http.ResponseWriter
```


is conceptually like:

```
ResponseWriter w = new Response();
```

where `Response` is Go's internal implementation.

So your understanding is:

Go Type	What it is
<code>*http.Request</code>	Pointer to a concrete struct (object)
<code>http.ResponseWriter</code>	Interface (abstraction/contract)

That's exactly the right mental model.

One small correction

When we say "object" in Go, it's usually more accurate to say **"struct instance"** or **"value of a struct type"**, because Go isn't a traditional object-oriented language like Java or C++. But if you're using "object" as a learning bridge from Java, that's a perfectly reasonable way to think about it initially.

YES!! 🎉 This is exactly how interfaces work in Go.

You're reaching one of the biggest "aha!" moments.

Let's use your example.

Your handler receives:

```
func handler(w http.ResponseWriter, r *http.Request)
```

The type of `w` is

```
http.ResponseWriter
```

which is an interface.

Suppose internally Go does something like this (simplified):

```
type response struct {  
    // internal fields  
}  
  
func (r *response) Header() Header {  
    fmt.Println("Header() called")  
    ...  
}  
  
func (r *response) Write(b []byte) (int, error) {  
    fmt.Println("Write() called")  
    ...  
}  
  
func (r *response) WriteHeader(code int) {  
    fmt.Println("WriteHeader() called")  
    ...  
}
```

Now imagine the server does

```
resp := &response{}  
  
handler(resp, request)
```

Notice something interesting...

Your handler expects

`http.ResponseWriter`

but receives

`*response`

because `*response` **implements** the interface.

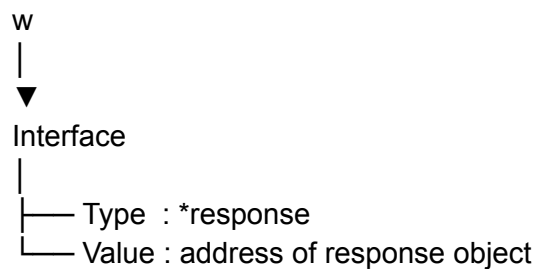
Now inside your handler

When you write

```
w.WriteHeader(200)
```

Go looks at the concrete value stored inside `w`.

Conceptually it's like:



So Go says

"The actual object is a `*response`. Let me call its `WriteHeader()` method."

It becomes

```
resp.WriteHeader(200)
```

Similarly,

```
w.Write([]byte("Hello"))
```

actually executes

```
resp.Write([]byte("Hello"))
```

Think of it like a TV remote

Suppose you have this interface:

```
type TV interface {  
    PowerOn()  
}
```

Now two companies implement it.

```
type SonyTV struct{}
```

```
func (s SonyTV) PowerOn() {  
    fmt.Println("Sony turned on")  
}
```

```
type LGTV struct{}
```

```
func (l LGTV) PowerOn() {  
    fmt.Println("LG turned on")  
}
```

Now

```
var tv TV
```

could contain either

```
tv = SonyTV{}
```

or

```
tv = LGTV{}
```

When you do

```
tv.PowerOn()
```

Go automatically calls the correct implementation.

If it's

```
SonyTV{}
```

it runs

```
SonyTV.PowerOn()
```

If it's

```
LGTV{}
```

it runs

```
LGTV.PowerOn()
```

You don't need an **if** statement.

Exactly the same with **ResponseWriter**

```
w.Header()
```

↓

calls the concrete object's

```
Header()
```

```
w.WriteHeader(201)
```

↓

calls

```
WriteHeader(201)
```

of the concrete implementation.

```
w.Write(data)
```

↓

calls

Write(data)

of the concrete implementation.

This is called dynamic dispatch

The decision of **which implementation** to execute is made **at runtime**, based on the concrete value stored inside the interface.

One tiny correction to your wording

You said:

"whoever has implement this interface will execute"

Almost. More precisely:

The concrete object currently stored inside the interface executes its implementation of the method.

This distinction matters because **many different types** can implement the same interface, but at runtime the interface variable holds **exactly one concrete value**. That's the one whose methods are called.

This understanding will be incredibly useful when you start using `io.Reader`, `io.Writer`, database interfaces, and dependency injection in Go. You're building the right mental model.

This is **the** question that makes Go's HTTP server click. Once you understand this flow, handlers become much less mysterious.

Let's go from the moment you type a URL in your browser.

Step 1: You start the server

Suppose your code is:

```
package main

import (
    "fmt"
    "net/http"
)

func hello(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Hello")
}

func main() {
    http.HandleFunc("/hello", hello)

    http.ListenAndServe(":8080", nil)
}
```

When this runs:

```
http.ListenAndServe(":8080", nil)
```

Go starts an HTTP server and waits.

Go Program



Listening on port 8080



Waiting...

Nothing happens until someone connects.

Step 2: Browser sends a request

You open:

`http://localhost:8080/hello`

The browser sends something like:

`GET /hello HTTP/1.1`

`Host: localhost:8080`

`User-Agent: Chrome`

This request reaches the Go server.

Browser

|

`GET /hello`

|



Go HTTP Server

Step 3: Go creates the Request object

The Go HTTP server reads the raw HTTP message.

It creates a struct similar to:

```
request := &http.Request{
    Method: "GET",
    URL: ...
    Header: ...
    Body: ...
}
```

This is your `r`.

Notice:

You never create it.

Go creates it for you.

Step 4: Go creates the Response object

The browser is waiting for a response.

So Go also creates an internal response object.

Conceptually:

```
response := &response{}
```

This type is internal to the `net/http` package.

It implements:

```
type ResponseWriter interface {  
    Header() Header  
    Write([]byte) (int, error)  
    WriteHeader(statusCode int)  
}
```

This is your `w`.

Again,

You never create it.

Step 5: Go matches the route

Earlier you wrote:

```
http.HandleFunc("/hello", hello)
```

Internally, Go stores something like:

```
"/hello"  
|
```

▼
hello()

Now the server sees

GET /hello

It finds

"/hello"

and decides

"I should call `hello`."

Step 6: Go calls your handler

Conceptually, the server executes:

hello(response, request)

or more accurately:

hello(w, r)

where

w

contains

internal response object

and

r

contains

`http.Request`

You never write this call.

Go calls your function.

Step 7: Your handler runs

Now you're inside:

```
func hello(w http.ResponseWriter, r *http.Request) {  
  
}
```

You can read from `r`.

`r.Method`

`r.URL.Path`

`r.Body`

And you can write to `w`.

`w.WriteHeader(200)`

`fmt.Fprintln(w, "Hello")`

Step 8: Your writes go into the response

When you write

```
fmt.Fprintln(w, "Hello")
```

it eventually calls

```
w.Write(...)
```

The internal response object stores

Status : 200

Headers:

Body:

Hello

Step 9: Go sends the response

After your handler returns,

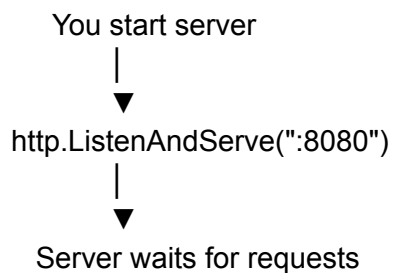
Go converts everything into HTTP.

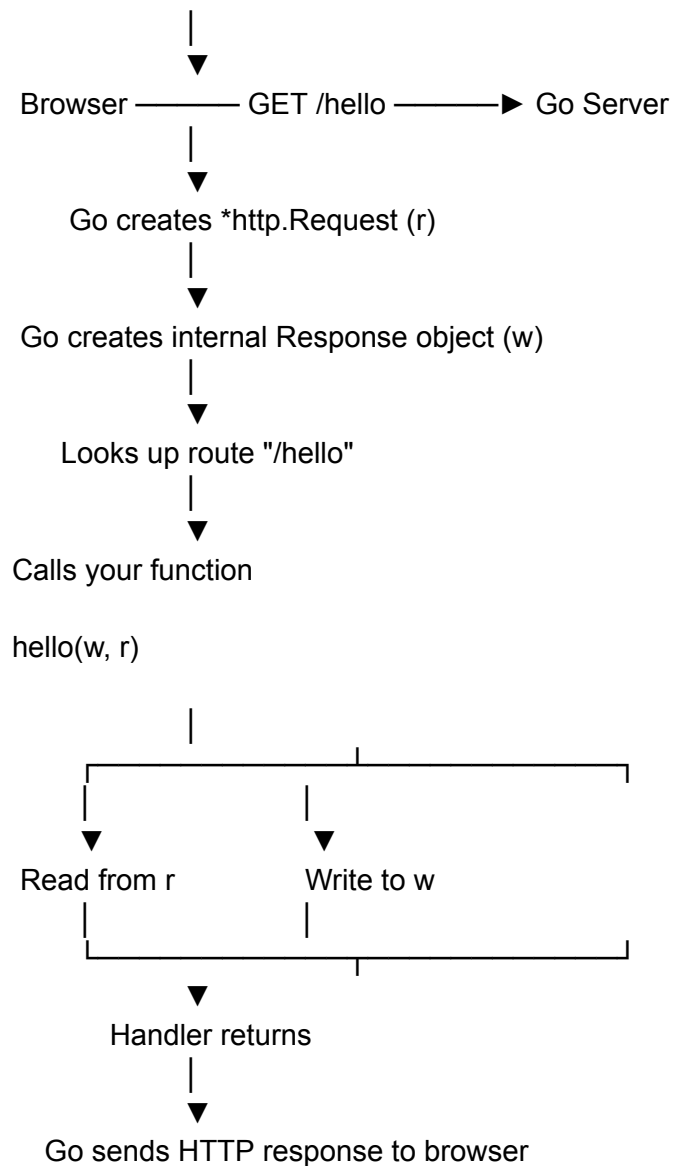
HTTP/1.1 200 OK

Hello

and sends it back.

Complete Flow





The biggest realization

Many beginners think **they call the handler**.

Actually, it's the opposite.

You register the handler:

```
http.HandleFunc("/hello", hello)
```

After that, **the Go HTTP server owns the control flow**.

When a request arrives, **Go** does something conceptually like this:

```
for {
    request := waitForIncomingRequest()

    r := buildRequest(request)
    w := buildResponseWriter()

    handler(w, r)
}
```

You don't create `r`.

You don't create `w`.

You don't call the handler.

The Go HTTP server creates both objects and invokes your handler, passing them in.

This is an example of the **Hollywood Principle** (also called **Inversion of Control**):

"Don't call us; we'll call you."

You provide the handler function, and the framework (`net/http`) decides **when** to invoke it. That's a pattern you'll see in many web frameworks across different languages.

These are **excellent questions**. You're no longer asking "what" Go does, you're asking **why** it is designed this way.

Let's tackle them one by one.

Question 1: Why `&http.Server{...}`?

You wrote:

```
server := &http.Server{
    Addr:    cfg.HttpServer.Addr,
    Handler: router,
}
```

and you're wondering why there's an `&`.

First, without the `&`:

```
server := http.Server{
    Addr:    cfg.HttpServer.Addr,
    Handler: router,
}
```

This creates a **Server value**.

Think of it as:

Stack

```
server
+-----+
| Addr    |
| Handler  |
| ...many fields...|
+-----+
```

With the `&`:

```
server := &http.Server{
    Addr:    cfg.HttpServer.Addr,
    Handler: router,
}
```

Go first creates the struct:

```
http.Server{
    Addr: ...
    Handler: ...
}
```

Then immediately takes its address.

So now:

server



0x1234

Heap

```
+-----+
| Addr   |
| Handler|
| ...many fields...|
+-----+
```

So `server` is of type

`*http.Server`

Why do people usually use a pointer?

Because most methods on `http.Server` are defined on a **pointer receiver**.

For example (simplified):

```
func (s *Server) ListenAndServe() error
```

Notice:

`*s`

not

`s Server`

This means the method expects a pointer.

Question 2: Earlier you wrote

I showed

```
http.ListenAndServe(":8080", nil)
```

Now you're using

```
server := &http.Server{
    Addr: cfg.HttpServer.Addr,
    Handler: router,
}
```

Why?

Because these are **two different ways of starting a server**.

Way 1 (Simple)

```
http.ListenAndServe(":8080", router)
```

This is just a convenience function.

Internally it's approximately:

```
server := &http.Server{
    Addr: ":8080",
    Handler: router,
}
```

```
server.ListenAndServe()
```

So this:

```
http.ListenAndServe(...)
```

is basically shorthand.

Way 2 (Explicit)

You create the server yourself.

```
server := &http.Server{
    Addr: ":8080",
    Handler: router,
}
```

```
server.ListenAndServe()
```

Now you can configure many more things.

For example

```
server := &http.Server{
    Addr:      ":8080",
    Handler:   router,
    ReadTimeout: 5 * time.Second,
    WriteTimeout: 10 * time.Second,
}
```

This is why production code usually uses `http.Server`.

Question 3 (This is the interesting one)

You said:

I feel Handler is just a register who registers the route and corresponding function?

You're **very close**, but there are actually **two different concepts**.

1. Router (or ServeMux)

This registers routes.

For example

```
mux := http.NewServeMux()

mux.HandleFunc("/hello", hello)
mux.HandleFunc("/users", users)
```

The router stores something like

```
"/hello" ---> hello()
```

```
"/users" ---> users()
```

Think of it as a map.

2. Handler

A handler is something that can **handle one HTTP request**.

The interface is

```
type Handler interface {
    ServeHTTP(ResponseWriter, *Request)
}
```

Notice it doesn't know anything about routes.

It only knows

```
"Give me a request, I'll handle it."
```

The router itself is actually a Handler!

For example

```
mux := http.NewServeMux()
```

Its type is

```
*http.ServeMux
```

And guess what?

It implements

`ServeHTTP(w, r)`

So

`mux`

is a Handler.

So the server does this

Incoming Request



Server



Handler (router)



Looks at URL



Calls `hello()`

So the server **doesn't know about routes**.

It only knows

"I have a Handler."

That Handler happens to be a router.

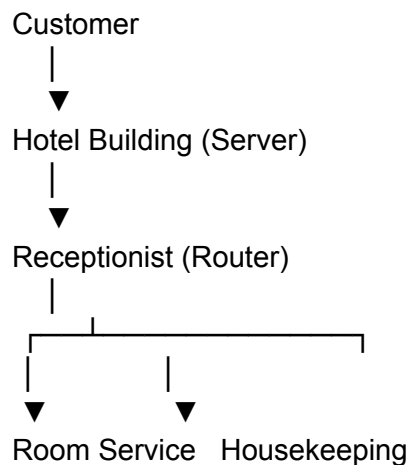
Analogy

Imagine a hotel.

The **server** is the hotel building.

The **router** is the receptionist.

The **handlers** are the different departments.



The server doesn't know where to send guests.

It simply asks the receptionist (the router).

The receptionist decides based on the request.

About **flag.String**

You compared it to

`flag.String(...)`

Not exactly.

flag.String registers a command-line flag.

It tells the `flag` package:

"If someone passes `-name`, store it here."

Similarly,

```
mux.HandleFunc("/hello", hello)
```

registers:

"If someone requests `/hello`, call `hello`."

So **the registration idea is similar**.

But remember:

- `HandleFunc` registers **routes** with a router.
- A `Handler` is **not** the registration. It's the thing that actually processes a request once the router has chosen it.

The key insight

The server only has one field:

Handler: router

because from the server's perspective:

"I don't care whether this is a router, middleware, reverse proxy, or something else. I just need **something that implements `ServeHTTP`**."

That's one of the elegant parts of Go's design: everything that can process an HTTP request implements the same simple `Handler` interface, so components can be composed very naturally.

This is one of those pieces of code that looks confusing until you know **what the validator package returns**. Let's build the mental model instead of just reading the code.

Step 1: What is `validator.New().Struct(student)` doing?

Suppose your struct is

```
type Student struct {  
    Name string `json:"name" validate:"required"`  
    Email string `json:"email" validate:"required"`  
    Age int `json:"age"`  
}
```

Now the client sends

```
{  
  "name": "Suyash"  
}
```

Notice

Email is missing.

Your code runs

```
err := validator.New().Struct(student)
```

The validator checks every field.

Student

```
|  
|— Name ✓ Present  
|— Email ✗ Missing  
|— Age ✓ OK
```

Since validation failed,

```
err != nil
```

Step 2: What is inside `err`?

Here's the important part.

`err` is of type

`error`

because `Struct()` returns

```
func Struct(s interface{}) error
```

But the **actual object** inside `err` is not just a simple error.

It is

`validator.ValidationErrors`

Think of it like this.

```
err
|
▼
error interface
|
▼
validator.ValidationErrors
```

Remember our discussion about interfaces?

Exactly the same thing.

Step 3: Why type cast?

Your code

```
validateErrs := err.(validator.ValidationErrors)
```

This is called a **type assertion**.

You're telling Go:

```
"I know this error is actually a ValidationErrors object."
```

After this,

```
validateErrs
```

is no longer just an **error**.

It becomes

```
validator.ValidationErrors
```

Now you can loop over it.

Imagine the validator found

Email -> required

Name -> max

Phone -> required

Internally

```
validateErrs
```

contains something like

```
[  
  ValidationError{Field:"Email", Tag:"required"},  
  ValidationError{Field:"Name", Tag:"max"},  
  ValidationError{Field:"Phone", Tag:"required"},  
]
```

]

Step 4: Passing it

Now you call

```
response.ValidationError(validateErrs)
```

Look at the function

```
func ValidationError(errs validator.ValidationErrors) Response
```

Notice it expects

```
validator.ValidationErrors
```

not

error

That's why you type-assert first.

Now let's understand the second function.

Step 1

```
func ValidationError(errs validator.ValidationErrors) Response {
```

Suppose

```
errs
```

contains

```
[  
    Email required,  
    Name required  
]
```

Step 2

```
var errMsgs []string
```

Creates an empty slice.

Initially

```
errMsgs
```

```
[]
```

Step 3

```
for _, err := range errs {
```

This loops through every validation error.

First iteration

```
err
```

```
Field = Email  
Tag = required
```

Second iteration

```
err
```

```
Field = Name  
Tag = required
```

Step 4

```
switch err.ActualTag() {
```

Every validation error knows which validation failed.

Example

```
validate:"required"
```

returns

required

If you had

```
validate:"email"
```

it returns

email

If

```
validate:"min=10"
```

returns

min

So

```
err.ActualTag()
```

asks

```
"Which validation rule failed?"
```

Step 5

Suppose

```
validate:"required"
```

failed.

Execution enters

```
case "required":
```

Now

```
fmt.Sprintf(  
    "field %s is required field",  
    err.Field(),  
)
```

Suppose

```
err.Field()
```

returns

Email

Result

```
field Email is required field
```

Then

```
append(errMsgs, ...)
```

adds it.

Initially

```
[]
```

becomes

```
[  
  "field Email is required field"  
]
```

Second iteration

```
[  
  "field Email is required field",  
  "field Name is required field"  
]
```

Step 6

Finally

```
strings.Join(errMsgs, ", ")
```

Suppose

errMsgs

contains

```
[  
  "field Email is required field",  
  "field Name is required field",  
]
```

`Join()` combines them.

Result

field Email is required field, field Name is required field

Step 7

Return

```
return Response{
    Status: StatusError,
    Error: strings.Join(errMsgs, ", "),
}
```

So

```
Response{
    Status: "Error",
    Error: "field Email is required field, field Name is required field",
}
```

Then WriteJson()

Remember

```
response.WriteJson(
    w,
    http.StatusBadRequest,
    response.ValidationError(validateErrs),
)
```

passes this struct

```
Response{
    Status: "Error",
    Error: "field Email is required field, field Name is required field",
}
```

to

```
json.NewEncoder(w).Encode(data)
```

The encoder converts it into

```
{  
  "status": "Error",  
  "error": "field Email is required field, field Name is required field"  
}
```

and sends it back to the client.

Complete flow

Client sends JSON



Decode into Student



validator.Struct(student)



Validation fails



Returns error interface



Actual type = ValidationErrors



Type assertion

err.(validator.ValidationErrors)



Loop through each failed field



Create readable messages



Join all messages



Create Response struct



Encode to JSON



Send HTTP 400 response

One thing I would improve in this code

Instead of writing:

```
validateErrs := err.(validator.ValidationErrors)
```

you can safely write:

```
validateErrs, ok := err.(validator.ValidationErrors)
if !ok {
    response.WriteJson(w, http.StatusInternalServerError, response.GeneralError(err))
    return
}
```

This is called the **comma-ok type assertion**. It avoids a panic if, for some unexpected reason, `err` is not actually a `validator.ValidationErrors`. In practice, `validator.Struct()` usually returns that type for validation failures, but getting into the habit of safe type assertions is a good Go practice.

Good. Let's forget the validator package for a moment. **The confusion is actually about interfaces, not validation.**

Once you understand **type assertion**, the validator code will become obvious.

Imagine this code

```
type Animal interface {  
    Speak()  
}
```

```
type Dog struct{}
```

```
func (Dog) Speak() {  
    fmt.Println("Woof")  
}
```

Now,

```
var a Animal
```

At this moment,

`a` can hold **any object** that implements `Animal`.

Now we do

```
a = Dog{}
```

What is the type of `a`?

You might think

Dog

No.

The **variable type** is still

Animal

Think of it like this:

a

Type : Animal (interface)

Value : Dog{}

This is **VERY IMPORTANT**.

The variable's type and the value inside it are different things.

Now suppose Dog has another method

```
func (Dog) Run() {  
    fmt.Println("Running")  
}
```

Can you do

a.Run()

 No.

Why?

Because Go only knows

a is an Animal

Animal only has

Speak()

It doesn't know about `Run()`.

But YOU know

You know that inside `a` there is actually a `Dog`.

So you tell Go

```
dog := a.(Dog)
```

Read it like English:

"Go, I know that inside this interface there is actually a `Dog`."

This is called a **type assertion**.

Now

```
dog.Run()
```

works.

Now let's relate it to your validator code.

The function returns

```
err
```

whose type is

```
error
```

Remember

```
error
```

is also an interface.

Exactly like `Animal`.

Internally

err

Type : error

Value : ValidationErrors

Exactly like

a

Type : Animal

Value : Dog

Now your code says

```
validateErrs := err.(validator.ValidationErrors)
```

Read it like English:

"Go, I know this error is actually a ValidationErrors."

Now `validateErrs` has the concrete type

`validator.ValidationErrors`

and you can iterate over it.

Now your second question

Casting vs Type Assertion

People often mix them up.

They are completely different.

1. Type Casting (Conversion)

Example

```
var x int = 10
```

```
var y float64 = float64(x)
```

Here

int

becomes

float64

We're converting one type into another.

Another example

```
var x int = 65
```

```
string(rune(x))
```

becomes

"A"

We're converting.

No interface involved.

2. Type Assertion

Only works on **interfaces**.

Suppose

```
var x interface{}
```

Now

```
x = "Hello"
```

So

Variable Type

```
interface{}
```

Actual Value

```
"Hello"
```

Now

```
s := x.(string)
```

You're **not converting** anything.

You're simply telling Go

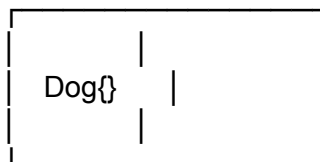
"Please give me the string that's already inside."

Nothing changes.

You're just revealing the real type.

Think of it like a gift box.

```
interface{}
```



Type assertion is opening the box.

```
Dog{}
```

Casting is different.

It's like melting iron into steel.

You're creating another representation.

The easiest way to remember

Casting

```
int
```

↓

```
float64
```

Convert one type into another.

Assertion

```
interface{}
```

↓

```
Dog
```

Reveal what's already inside the interface.

One last question for you

After reading this, what do you think this means?

```
validateErrs := err.(validator.ValidationErrors)
```

Try answering in your own words.

Don't worry about being perfectly correct. This is the point where your understanding will become much stronger.

YES!! 🎉 You're about **95% there**. I just want to refine two small points.

You said:

the error is type of Error interface

✅ Correct.

More precisely:

```
var err error
```

means the **variable type** is **error** (an interface).

Then you said:

but it has validator.ValidationErrors value inside

✅ Exactly.

Think of the interface like this:

```
err
|
|— Variable Type : error
|
|— Actual Value : validator.ValidationErrors
```

Just like we discussed with **Animal** and **Dog**.

Then you said:

so give me the `validator.ValidationErrors`

YES!

That's literally what this line says:

```
validateErrs := err.(validator.ValidationErrors)
```

Read it as English:

"Go, I know this `error` interface actually contains a `validator.ValidationErrors`. Please give me that concrete value."

Now your last sentence:

so that I can iterate, as it is like array like data structure, but it is not an array

Very close.

It is **not an array**, but it's also not just "array-like".

If you look at the validator package, you'll find something like this (simplified):

```
type ValidationErrors []FieldError
```

Notice this:

```
[]FieldError
```

That means **`ValidationErrors` is a named slice type**.

It's essentially:

```
type ValidationErrors = []FieldError
```

except it's a **new named type**, not an alias.

So this works:

```
for _, err := range validateErrs {  
    // ...  
}
```

because **slices are iterable**, and `ValidationErrors` is a slice type.

Think of it like this:

`validator.ValidationErrors`

↓

`[]FieldError`

↓

```
[  
    FieldError,  
    FieldError,  
    FieldError,  
]
```

Each `FieldError` represents one validation failure.

For example:

```
{  
    "name": "",  
    "email": ""  
}
```

might produce:

```
[  
    Name is required,  
    Email is required,  
]
```

So the loop:

```
for _, err := range validateErrs {
```

iterates over each `FieldError`.

Your understanding, rewritten slightly

I'd phrase it like this:

"`err` is an `error` interface variable. The actual value stored inside it is a `validator.ValidationErrors`. Using the type assertion `err.(validator.ValidationErrors)`, I extract the concrete `ValidationErrors` value so I can use its slice behavior (like `range`) and any methods defined on it."

That explanation is **software engineer interview level**. It shows you understand the difference between an interface's **static type** and its **dynamic value**.

You're asking exactly the kinds of questions that build a strong understanding of Go's type system.

Excellent. This is a Go language feature, not a validator feature.

There are **two ways** to create a type in Go.

1. Create a NEW named type

```
type Age int
```

Read it as:

"Create a **new type** called `Age`, whose underlying type is `int`."

Now:

```
var a Age = 23  
var b int = 23
```

Can you do this?

```
b = a
```

✗ No.

Compiler error.

Why?

Because

Age

and

int

are **different types**, even though they both store integers.

You must convert:

```
b = int(a)
```

Another example

```
type StudentNames []string
```

Now

```
var names StudentNames
```

is **not** the same type as

```
var s []string
```

Even though internally both are slices of strings.

2. Create a type alias

Notice the `=`.

```
type Age = int
```

Now `Age` is **just another name** for `int`.

It's not a new type.

Now

```
var a Age = 23  
var b int = 23
```

This works:

```
b = a
```

No conversion needed.

Because

`Age`

and

`int`

are literally the same type.

Visual Difference

New type

```
type Age int
```

int
|
▼
Age

Different types

Alias

type Age = int

Age
|
▼
int

Exactly the same type

Now your validator example

The package defines something like

```
type ValidationErrors []FieldError
```

Notice

There is **NO** `=`.

That means

`ValidationErrors` is its own type.

Its underlying type is

```
[]FieldError
```

So internally it looks like

ValidationErrors

↓

Underlying type

↓

[]FieldError

Why create a new type?

Because now the package can attach methods to it.

Example

```
type StudentNames []string
```

Now you can do

```
func (s StudentNames) Print() {  
    ...  
}
```

You **cannot** do that with

```
[]string
```

because Go doesn't let you add methods to built-in types.

That's one of the biggest reasons packages create new named types.

Small Exercise

Tell me whether these are **new types** or **aliases**.

A

```
type Height float64
```

B

```
type Height = float64
```

C

```
type ValidationErrors []FieldError
```

Try answering all three. If you get them right, you've understood one of the most important parts of Go's type system.

🎉 **100% correct.**

Let's verify them.

A

```
type Height float64
```

✅ **New named type**

- Name: `Height`
- Underlying type: `float64`

These are **different types**:

```
var h Height = 180.5  
var f float64
```

```
// f = h    ❌ Compile error  
f = float64(h) // ✅ Conversion required
```

B

```
type Height = float64
```

✓ Type alias

`Height` is just another name for `float64`.

```
var h Height = 180.5
var f float64
```

```
f = h // ✓ Works
```

No conversion is needed because they are literally the same type.

C

```
type ValidationErrors []FieldError
```

✓ New named type

Underlying type:

```
[]FieldError
```

But the actual type is

```
ValidationErrors
```

That's why the validator package can define methods on it.

For example (simplified):

```
func (v ValidationErrors) Error() string {
    // ...
}
```

If `ValidationErrors` were just an alias, it wouldn't be a distinct named type to build an API around.

One more thing that will make everything click

When you see:

```
type ValidationErrors []FieldError
```

Don't read it as:

"**ValidationErrors** is a slice."

Read it as:

"**ValidationErrors** is a new type whose underlying representation is a slice of **FieldError**."

That's why it supports things like:

```
for _, err := range validateErrs {  
    fmt.Println(err.Field())  
}
```

because its underlying representation is a slice, so **range** works on it.

This is a good sign

A few messages ago, you were asking:

"What is **NewDecoder**?"

Now you're asking about:

- Interfaces vs concrete types
- Type assertions
- New named types vs aliases
- Why libraries design their APIs this way

These are the kinds of questions that build a deep understanding of Go. They're also the concepts that interviewers often probe because they reveal whether someone understands the language beyond just writing syntax.

Perfect. This is where many people get confused because they read it as **one expression**.

Don't.

Break it into **three steps**.

Instead of reading

```
validator.New().Struct(student)
```

read it as

```
v := validator.New()
```

```
err := v.Struct(student)
```

Now let's understand each part.

Step 1: What is **validator**?

You imported

```
import "github.com/go-playground/validator/v10"
```

Think of **validator** as a package.

Like

```
fmt.Println()
```

or

```
json.NewDecoder()
```

Here

```
validator.New()
```

is calling a function from the package.

Step 2: What is `New()`?

Inside the package (simplified), there is something like

package validator

```
type Validate struct {  
    // internal configuration  
}
```

```
func New() *Validate {  
    return &Validate{}  
}
```

Notice what it returns:

`*Validate`

A pointer to a `Validate` object.

So

```
v := validator.New()
```

means

```
v  
|  
▼
```

`*Validate`

Just like

```
server := &http.Server{}
```

returns

`*http.Server`

Step 3: What is `Struct()`?

The `Validate` type has a method.

Something like

```
func (v *Validate) Struct(s interface{}) error {  
    // validation logic  
}
```

Notice

`(v *Validate)`

This means

`Struct()` belongs to the `Validate` object.

So when you write

```
v := validator.New()
```

you now have

```
v.Struct(student)
```

Exactly like

```
server.ListenAndServe()
```

or

```
decoder.Decode(&student)
```

So this line

```
validator.New().Struct(student)
```

is simply chaining two operations.

Instead of

```
v := validator.New()
```

```
err := v.Struct(student)
```

the author wrote

```
err := validator.New().Struct(student)
```

Because **v** is only used once.

Visual Flow

validator package



New()



returns



*Validate



Struct(student)



error

Compare it with **json.NewDecoder()**

You already learned

```
json.NewDecoder(r.Body).Decode(&student)
```

Let's compare them.

JSON

```
decoder := json.NewDecoder(r.Body)
```

```
err := decoder.Decode(&student)
```

or

```
json.NewDecoder(r.Body).Decode(&student)
```

Validator

```
v := validator.New()
```

```
err := v.Struct(student)
```

or

```
validator.New().Struct(student)
```

They follow **exactly the same pattern**.

Package



▼
New(...)



returns an object



▼
Call method on that object

Why `New()`? Why not just `validator.Struct(student)`?

Because the validator object can hold configuration.

For example, imagine the package looked like this:

```
v := validator.New()
```

```
v.RegisterValidation("phone", phoneValidator)
```

```
v.RegisterValidation("pincode", pinValidator)
```

```
err := v.Struct(student)
```

Now **your validator object** knows about your custom validation rules.

If `Struct()` were just a package function, there would be nowhere to store that configuration.

That's why many Go packages follow this design:

- `json.NewDecoder(...)`
- `json.NewEncoder(...)`
- `validator.New()`
- `http.NewServeMux()`

They create an object that may contain state or configuration, and then you call methods on that object.

A question for you

Based on what we've discussed, what do you think is the type of the value returned by:

```
validator.New()
```

Don't guess from memory—reason it out from the pattern.

🎉 **Exactly!** You've made the connection.

Yes, **New()** is Go's equivalent of a constructor, although Go doesn't have constructors as a language feature.

In Java

You write:

```
Validator v = new Validator();
```

The **new** keyword allocates memory and calls the constructor.

In Go

There is no **new Validator()** syntax.

Instead, package authors usually write a function named **New()**.

For example:

```
v := validator.New()
```

Internally (simplified), it's something like:

```
type Validate struct {  
    // configuration  
}
```

```
func New() *Validate {  
    return &Validate{}  
}
```

So yes, you can think of it as:

"Construct a Validate object and return its pointer."

This pattern is everywhere in Go

JSON Decoder

```
decoder := json.NewDecoder(r.Body)
```

Internally:

```
type Decoder struct {  
    reader io.Reader  
}  
  
func NewDecoder(r io.Reader) *Decoder {  
    return &Decoder{  
        reader: r,  
    }  
}
```

Then:

```
decoder.Decode(&student)
```

HTTP Server

```
server := &http.Server{}
```

or some packages provide:

```
server := http.NewServer(...)
```

Router

```
mux := http.NewServeMux()
```

Internally:

```
type ServeMux struct {  
    routes map[string]Handler  
}
```

```
func NewServeMux() *ServeMux {  
    return &ServeMux{  
        routes: make(map[string]Handler),  
    }  
}
```

Why not just use `new()`?

Go actually has a built-in function called `new`.

Example:

```
p := new(Person)
```

is equivalent to:

```
p := &Person{}
```

But package authors **rarely expose `new()` directly** because they usually want to do more than just allocate memory.

For example:

```
func New() *Validate {  
    return &Validate{  
        cache: make(map[string]Rule),  
        tagName: "validate",  
    }  
}
```

A constructor can:

- Initialize maps
- Set default values
- Allocate internal resources
- Register default configurations
- Return an interface instead of a concrete type (in some designs)

So `New()` is much more flexible than `new()`.

Your mental model

When you see:

```
validator.New()
```

don't think:

"Some magic function."

Think:

validator package



New()



Creates a Validate object



Returns *Validate



v

Then:

```
v.Struct(student)
```

calls a method on that object.

One small terminology correction

You said:

"it is Validator pointer"

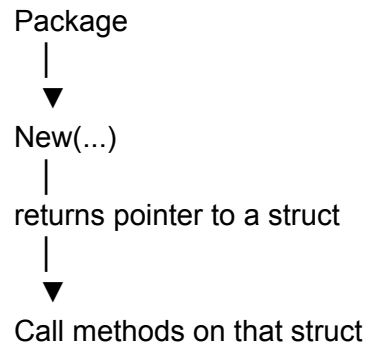
Almost. The type is a pointer to the package's `Validate` type, so it's:

```
*validator.Validate
```

not `*validator.Validator`.

That distinction matters because the struct is named `Validate`, while the package is named `validator`.

You're starting to recognize a very common Go design pattern:



You'll see this repeatedly in the standard library and popular Go packages.